HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY, VNU-HCM
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Deep Learning and Its Applications (CO3133)

Semester 261

Assignment 1 Report

Foundations of Deep Learning Pipelines and Architectures

From Linear Models to Modern Sequence Models:
A Comparative Study for Image Classification

Group Placeholder

Instructor	Le Thanh Sach
Students	**Student ID**
Tran Lam Anh	2352067
Vu Duc Viet Anh	2352074
Tran Nguyen Giap	2352284
Le Nguyen Khang	2352470

Ho Chi Minh City, 28 October 2026

Abstract

We compare Linear, MLP, CNN, GRU, LSTM, and Transformer classifiers on Fashion-MNIST to examine how image representation relates to accuracy and classification errors. Each model trains for 50 epochs with seed 42, using the same data split and preprocessing. Checkpoints minimize validation loss and are evaluated on the official 10,000-image test set. CNN achieves the highest accuracy, 92.59%, and a macro-F1 of 0.9261. It exceeds the Transformer by 2.16 percentage points while using 421,738 parameters rather than 1,191,050. Class recall and confusion matrices show that upper-body garments remain frequent sources of error. CNN identifies 79.5% of Shirt images, compared with 62.1% for MLP. Shared-image comparisons show that each model also correctly classifies examples the other misses. The results support the CNN configuration for this dataset, but do not establish a general ranking of architecture families. Each configuration uses one seed, and architecture, capacity, and regularization vary together. Recorded timings do not support a speed comparison because hardware and measurement conditions differ.

Table of Contents

Abstract **i**

1 Problem and data description **1**

1.1 Task and research question 1

1.2 Dataset and study scope 1

1.3 Exploratory data analysis 2

1.4 Splits and leakage prevention 3

1.5 Preprocessing and augmentation 3

2 Methodology **4**

2.1 Training and evaluation pipeline 4

2.2 Model architectures 4

2.3 Architecture comparison 8

2.4 Loss and evaluation metrics 8

2.5 Training configuration 9

2.6 Comparison protocol 10

3 Implementation results **11**

3.1 Test performance and model size 11

3.2 Learning behavior 11

3.3 Per-class performance and error analysis 11

3.4 Predictions on shared images 13

3.5 Architecture trade-offs 14

3.6 Limitations 15

3.7 Conclusion 15

AI usage disclosure **16**

References **16**

List of Figures

1 Fashion-MNIST: the first image of each class in official training order, selected without using model predictions. . . . 2
2 Cosine similarity between centred Fashion-MNIST class-average images, computed from the 60,000 official training images. . . . 2
3 Linear classifier. The flattened image is mapped to ten class logits. . . . 4
4 MLP with hidden widths 256 and 128, ReLU, and dropout. . . . 5
5 CNN for MNIST and Fashion-MNIST. Flattened convolutional features feed a 128-unit hidden layer and the classification head. . . . 5
6 GRU cell and row-sequence classifier. The final hidden state supplies the class logits. 6
7 Implemented LSTM cell and row-sequence classifier. . . . 7
8 Transformer with row embeddings, positional encoding, two encoder layers, and mean pooling. . . . 7
9 Fashion-MNIST training and validation cross-entropy for six models. Dots mark the selected checkpoints. . . . 12
10 Linear and MLP confusion matrices on the Fashion-MNIST test set. Rows are true classes and columns are predictions. Color shows row proportions; annotations show counts. . . . 13
11 CNN and GRU confusion matrices on the Fashion-MNIST test set. Rows are true classes and columns are predictions. Color shows row proportions; annotations show counts. . . . 13
12 LSTM and Transformer confusion matrices on the Fashion-MNIST test set. Rows are true classes and columns are predictions. Color shows row proportions; annotations show counts. . . . 14
13 Shared Fashion-MNIST test images selected by CNN/MLP disagreement. Each image has the same five model predictions. Baseline denotes MLP. . . . 14

List of Tables

1 Dataset roles, native image sizes, and split counts. Image dimensions are height × width × channels. Counts refer to images. . . . 1
2 Classifier dimensions and trainable parameters. MNIST and Fashion-MNIST share the same shapes. . . . 8
3 Training settings for the reported runs. . . . 9
4 Fashion-MNIST test results. Each model trains for 50 epochs with seed 42; checkpoints minimize validation loss. . . . 11
5 Fashion-MNIST checkpoint selection from saved training summaries. . . . 11
6 Fashion-MNIST test recall by class, in percent. Each class contains 1,000 images. All six models use seed 42 and validation-selected checkpoints from 50-epoch runs. 12

1 Problem and data description

1.1 Task and research question

The task is supervised image classification: assign one clothing category to each image. Category prediction can help organize online product catalogs. The study asks how linear, multilayer, convolutional, recurrent, and Transformer models differ in accuracy and class confusions on the same data splits.

For Fashion-MNIST, the model receives a normalized grayscale tensor $x \in \mathbb{R}^{1\times28\times28}$ and returns ten class scores $f_\theta(x) \in \mathbb{R}^{10}$. Its prediction is $\hat{y} = \arg\max_{k\in\{0,\dots,9\}} f_\theta(x)_k$. The prediction unit is one image. Low resolution and similar garment outlines make some categories difficult to separate.

1.2 Dataset and study scope

Fashion-MNIST contains 70,000 Zalando product images with supplied category labels: 60,000 official training images and 10,000 official test images [3]. Labels 0–9 denote T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, and Ankle boot, respectively.

We use the standard IDX distribution from the authors' MIT-licensed repository [4]. The four cached gzip files total 30.88 MB and match the published MD5 checksums. Images are unsigned 8-bit arrays; labels are integer class IDs. No samples are removed or relabelled. The small, isolated product images and fixed category set limit how far results extend to full scenes or unfamiliar products.

Fashion-MNIST is the main comparison dataset. MNIST handwritten digits are used for development, and CIFAR-10 colour images provide a separate extension [5, 6]. Their roles and working splits appear in Table 1.

Table 1: Dataset roles, native image sizes, and split counts. Image dimensions are height × width × channels. Counts refer to images.

Dataset	Role	Image size	Train	Validation	Test
Fashion-MNIST	Main comparison	$28 \times 28 \times 1$	54,000	6,000	10,000
MNIST	Development	$28 \times 28 \times 1$	54,000	6,000	10,000
CIFAR-10	Extension	$32 \times 32 \times 3$	45,000	5,000	10,000

1.3 Exploratory data analysis

The following statistics use the full, unaugmented official Fashion-MNIST training set, before the train/validation split. Every image is 28×28 pixels. Each class has 6,000 images, so the largest-to-smallest class ratio is 1.00. The official test set has 1,000 images per class. Figure 1 shows one real example per class.

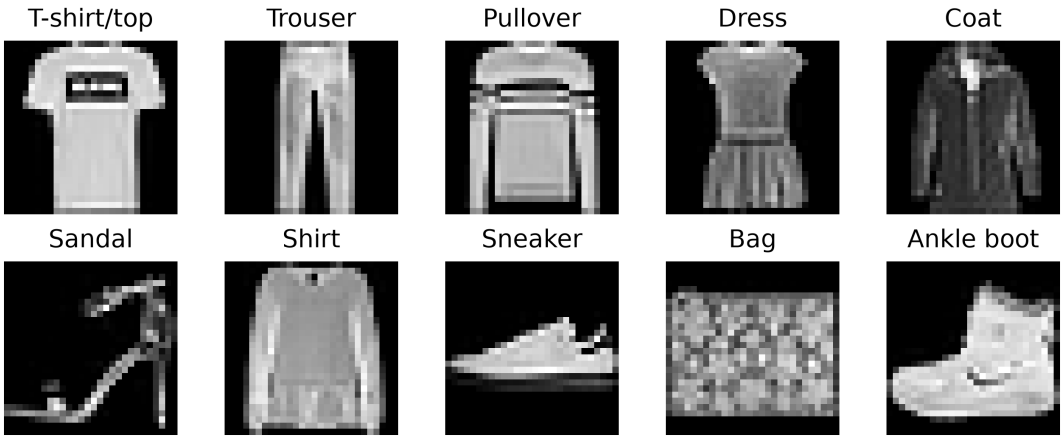


Figure 1: Fashion-MNIST: the first image of each class in official training order, selected without using model predictions.

Pixel intensity has mean 72.94 and standard deviation 90.02 on the 0–255 scale. About 53.0% of pixels fall in the lowest histogram bin (intensities 0–7), reflecting the dark background. Defining foreground as intensity above 32 gives average coverage from 24.5% for Sandal to 59.8% for Pullover.

Figure 2 compares class-average images after subtracting the average across classes. Cosine similarity is 0.86 for Pullover–Coat and 0.82 for both Sandal–Sneaker and Pullover–Shirt. These pairs motivate later inspection of class confusions; prototype similarity does not measure model errors.

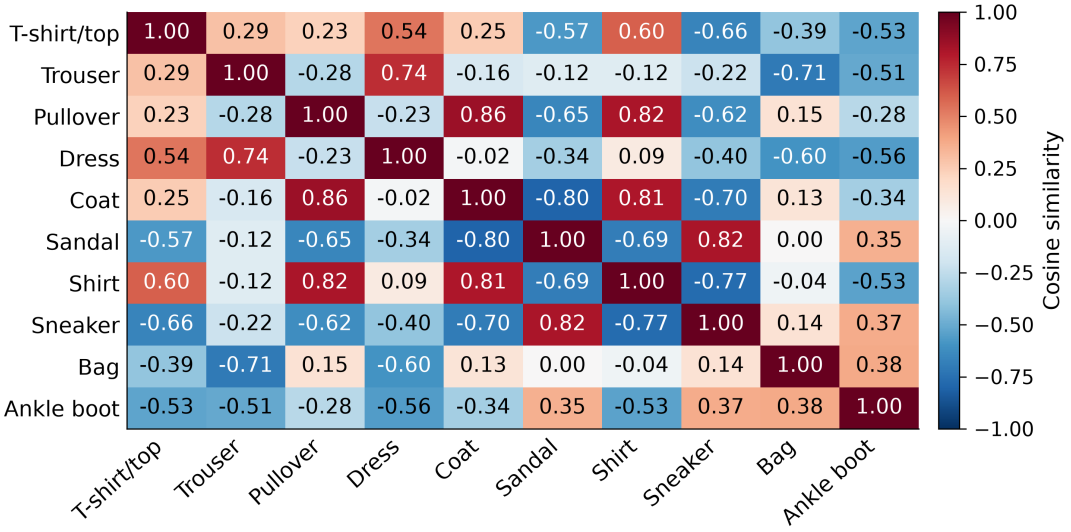


Figure 2: Cosine similarity between centred Fashion-MNIST class-average images, computed from the 60,000 official training images.

1.4 Splits and leakage prevention

For each dataset, a random permutation with seed 42 reserves 10% of the official training set for validation. The split unit is one image, and sampling is not stratified. For Fashion-MNIST, class counts therefore range from 5,353 to 5,445 in training and from 555 to 647 in validation. All models use the same indices for a given dataset.

The official test sets remain separate from training and validation. Validation loss selects checkpoints; test results are reserved for reporting. Training and validation indices are disjoint. No additional duplicate-image or product-level audit is recorded, so this separation does not establish independence between all images.

1.5 Preprocessing and augmentation

MNIST and Fashion-MNIST keep their native resolution. Pixels are converted to float32 in $[0, 1]$, then normalized as $x' = (x - 0.5)/0.5$. Training adds random rotation within $\pm 10^\circ$ and translation up to 10% of image size.

CIFAR-10 uses four-pixel padding, a 32×32 random crop, and a random horizontal flip for training. We normalize its RGB channels using mean $(0.4914, 0.4822, 0.4465)$ and standard deviation $(0.2470, 0.2435, 0.2616)$. Validation and test data receive conversion and normalization only.

An inspected Fashion-MNIST validation batch has float32 image shape $[128, 1, 28, 28]$ and values from -1 to 1. Its int64 labels have shape $[128]$; the first eight IDs are $[9, 2, 5, 0, 5, 1, 3, 5]$. In the shared $[B, C, H, W]$ convention, B is batch size, C is channels, and H, W are image height and width. CIFAR-10 batches use shape $[B, 3, 32, 32]$.

2 Methodology

2.1 Training and evaluation pipeline

The study compares classifiers trained from scratch on MNIST, Fashion-MNIST, and CIFAR-10. For each dataset, a seeded random permutation reserves 10% of its official training set for validation. The split is shared across models and is not stratified. This gives 54,000 training and 6,000 validation images for each grayscale dataset, and 45,000 training and 5,000 validation images for CIFAR-10. Each official test set contains 10,000 images.

During training, minibatches pass through the encoder and a linear classification head. Cross-entropy supplies the gradients for parameter updates. Dropout and batch normalization use training mode. After each epoch, the model is evaluated on the validation set with dropout disabled and batch normalization using its running statistics. The checkpoint with the lowest validation loss supplies the test predictions. The test set is excluded from checkpoint selection.

2.2 Model architectures

Each model maps an image to a feature vector $h \in \mathbb{R}^d$ and then computes ten logits, $z = Wh + b$. The following descriptions use Fashion-MNIST shapes. MNIST uses the same dimensions; CIFAR-10 supplies three-channel 32×32 images. Table 2 summarizes feature dimensions and parameter counts.

2.2.1 Linear classifier

The linear model flattens an image to $x \in \mathbb{R}^{784}$ and computes $z = Wx + b$, where $W \in \mathbb{R}^{10\times784}$ and $b \in \mathbb{R}^{10}$. Each class score is an affine function of the pixels. Flattening preserves their fixed coordinate order; the model imposes no local connectivity or spatial weight sharing. It provides the simplest baseline in the study.

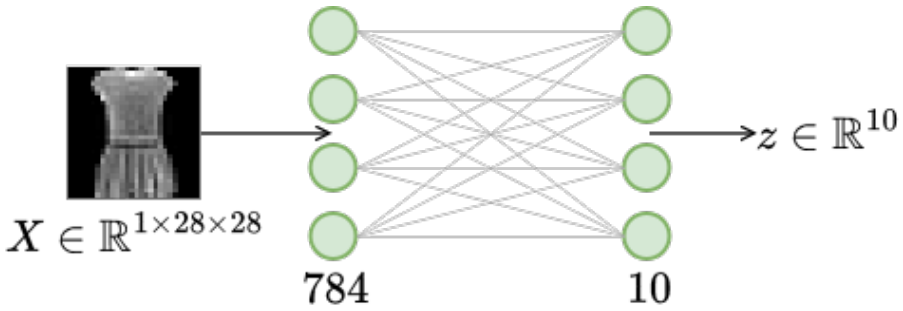


Figure 3: Linear classifier. The flattened image is mapped to ten class logits.

2.2.2 Multilayer perceptron

The MLP applies two hidden affine layers of widths 256 and 128 to the flattened image. Each hidden layer is followed by ReLU and dropout with probability 0.2. A linear head maps the 128-dimensional features to ten logits. The nonlinear layers allow interactions between pixel values, while connections remain fully connected without spatial weight sharing.

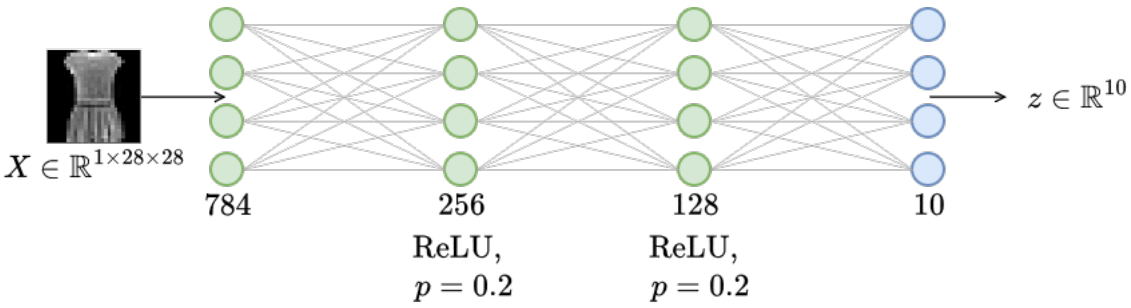


Figure 4: MLP with hidden widths 256 and 128, ReLU, and dropout.

2.2.3 Convolutional neural network

The CNN applies two convolutional blocks with 32 and 64 channels. Each block contains a 3×3 convolution with stride 1 and padding 1, batch normalization, ReLU, and 2×2 max pooling with stride 2. Convolution biases are disabled. For a 28×28 image, the blocks produce $32 \times 14 \times 14$ and $64 \times 7 \times 7$ feature maps. Flattening gives 3,136 values, followed by a 128-unit linear layer and ReLU. A ten-class linear head produces the logits. For CIFAR-10, the final feature map is $64 \times 8 \times 8$, so the hidden layer receives 4,096 values.

Convolution shares local filters across image positions. Flattening preserves the positions of their responses, and the hidden layer can combine responses across the full feature map. The classifier has 421,738 parameters for grayscale images and 545,194 for CIFAR-10.

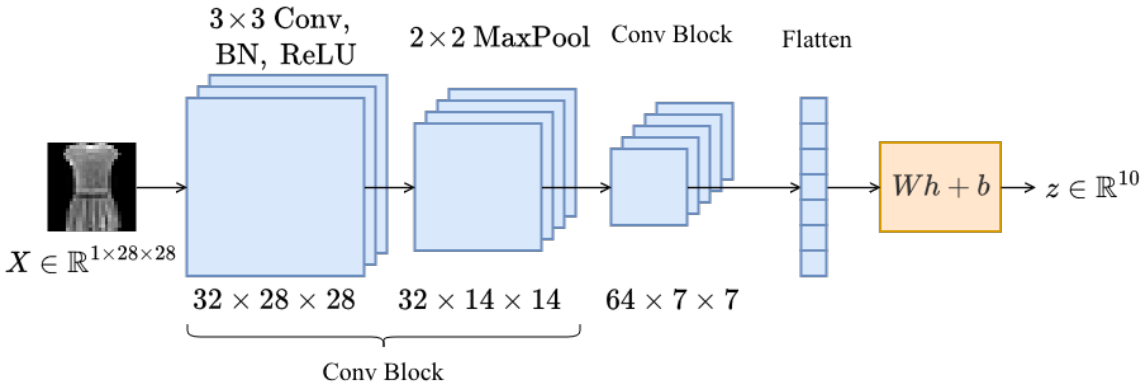


Figure 5: CNN for MNIST and Fashion-MNIST. Flattened convolutional features feed a 128-unit hidden layer and the classification head.

2.2.4 Gated recurrent unit

The GRU reads the image rows from top to bottom. An input of shape $C \times H \times W$ becomes a sequence of H vectors in $\mathbb{R}^{CW}$. Thus, Fashion-MNIST gives 28 steps with 28 values per step; CIFAR-10 gives 32 steps with 96 values per step.

The model has one recurrent layer with hidden size 128 and initial state $h_0 = 0$. Its updates are

$$\begin{aligned} r_t &= \sigma(W_{ir}x_t + b_{ir} + W_{hr}h_{t-1} + b_{hr}), \\ u_t &= \sigma(W_{iu}x_t + b_{iu} + W_{hu}h_{t-1} + b_{hu}), \\ \widetilde{h}_t &= \tanh(W_{in}x_t + b_{in} + r_t \odot (W_{hn}h_{t-1} + b_{hn})), \\ h_t &= (1 - u_t) \odot \widetilde{h}_t + u_t \odot h_{t-1}. \end{aligned}$$

Here, σ is the sigmoid function and $\odot$ denotes elementwise multiplication. The reset gate r_t controls the recurrent contribution to the candidate state; the update gate u_t controls retention of the preceding state. All steps share the same parameters. The final state h_H supplies the linear classifier. No recurrent dropout is used in the evaluated configuration.

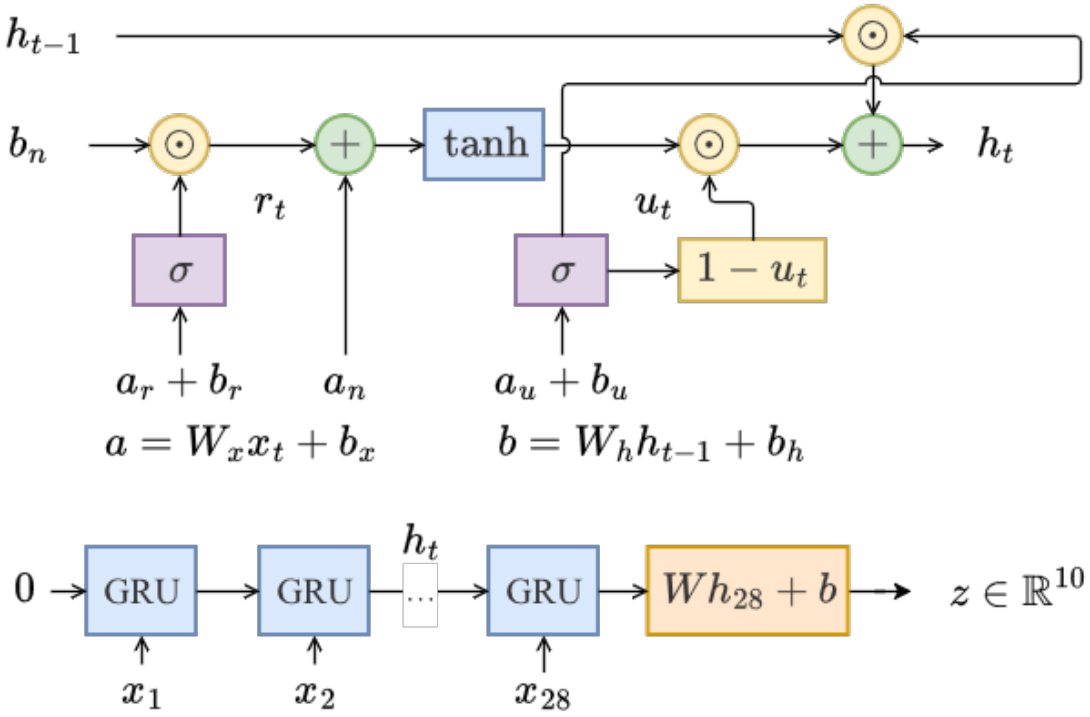


Figure 6: GRU cell and row-sequence classifier. The final hidden state supplies the class logits.

2.2.5 Long short-term memory

The repository also implements a one-layer LSTM with hidden size 128, using the same row sequence and a linear head on the final hidden state. In addition to h_t, it maintains a cell state c_t. With zero initial states, the updates are

$$\begin{aligned} q_t &= W_x x_t + b_x + W_h h_{t-1} + b_h, \\ (i_t, f_t, g_t, o_t) &= \big(\sigma(q_{t,i}), \sigma(q_{t,f}), \tanh(q_{t,g}), \sigma(q_{t,o})\big), \\ c_t &= f_t \odot c_{t-1} + i_t \odot g_t, \\ h_t &= o_t \odot \tanh(c_t), \end{aligned}$$

where $q_t \in \mathbb{R}^{512}$ is divided into four 128-dimensional vectors. The input, forget, and output gates control the cell update and hidden output. The LSTM is included in the measured comparisons.

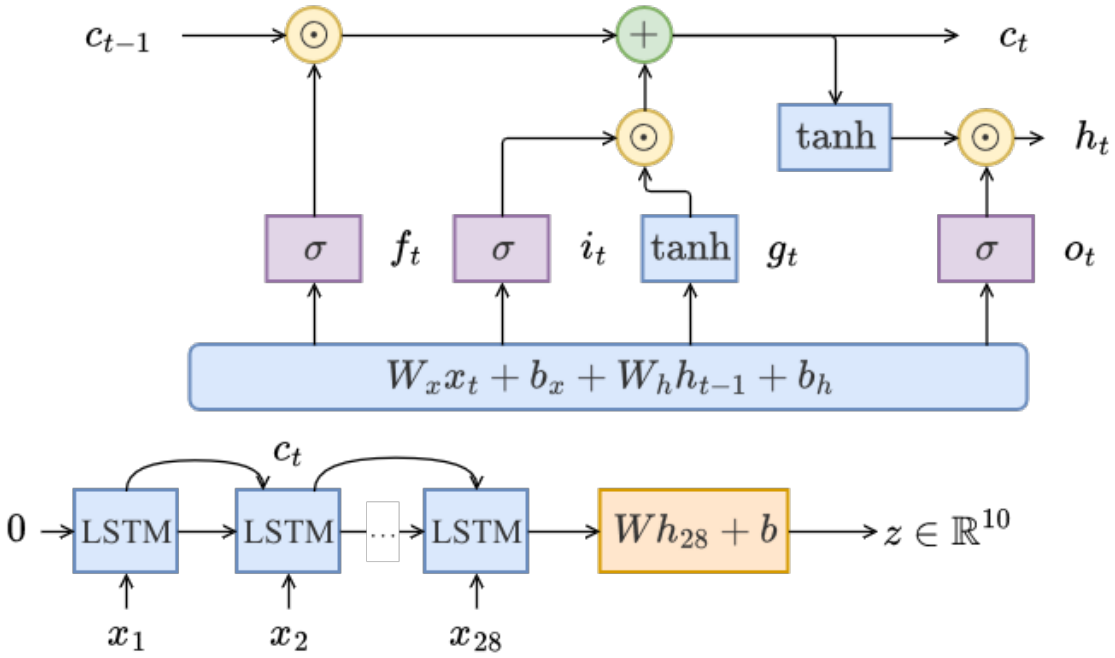


Figure 7: Implemented LSTM cell and row-sequence classifier.

2.2.6 Transformer

The Transformer uses the same row tokens as the recurrent models. Each token is projected to 128 dimensions and receives a sinusoidal positional encoding. Embedding dropout has probability 0.1. Two encoder layers each apply four-head self-attention and a feed-forward network of widths $128 \to 2048 \to 128$, with ReLU. Each sublayer uses a residual connection followed by layer normalization; encoder dropout is 0.1.

For a sequence representation $S \in \mathbb{R}^{H \times 128}$, one attention head computes

$$Q = SW_Q, \quad K = SW_K, \quad V = SW_V, \qquad \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{32}}\right) V, \quad (1)$$

where $W_Q, W_K, W_V \in \mathbb{R}^{128 \times 32}$ and the softmax acts across key positions. Projection biases are omitted from this notation. The four head outputs are concatenated and projected to 128 dimensions. The mean of the encoded row vectors supplies the linear classifier. Attention connects all row positions, while positional encoding supplies their order.

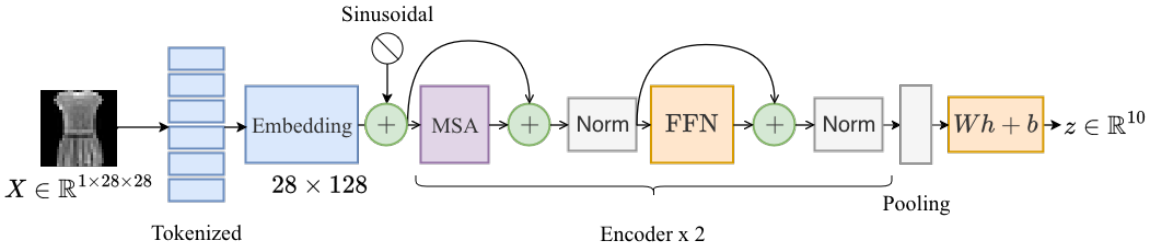


Figure 8: Transformer with row embeddings, positional encoding, two encoder layers, and mean pooling.

Table 2: Classifier dimensions and trainable parameters. MNIST and Fashion-MNIST share the same shapes.

Model	Feature dimension d	MNIST / Fashion-MNIST	CIFAR-10
Linear	784 / 3072	7,850	30,730
MLP	128	235,146	820,874
CNN	128	421,738	545,194
GRU	128	61,962	88,074
LSTM	128	82,186	117,002
Transformer	128	1,191,050	1,199,754

2.3 Architecture comparison

The models differ in representation, connectivity, and parameter count. Equal epoch counts therefore do not constitute matched capacity or compute. The comparison evaluates these complete configurations. It does not isolate the causal effect of any individual architectural component.

The group implements the model assembly, row-sequence processing, and GRU and LSTM cell updates. Affine layers, convolution, normalization, pooling, and the Transformer encoder use PyTorch modules. All classifiers are trained from scratch.

2.4 Loss and evaluation metrics

Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$ denote a labelled sample, where $y_i \in \{0, \ldots, K-1\}$ and $K = 10$. For an input x_i, the model produces a vector of logits $z_i = f_\theta(x_i) \in \mathbb{R}^K$. The corresponding class probabilities and predicted label are

$$p_{ik} = \frac{\exp(z_{ik})}{\sum_{j=0}^{K-1} \exp(z_{ij})}, \qquad \hat{y}_i = \arg\max_k z_{ik}. \tag{2}$$

Training objective. All models minimize the mean categorical cross-entropy,

$$\mathcal{L}_{\mathrm{CE}}(\theta; \mathcal{D}) = -\frac{1}{N} \sum_{i=1}^{N} \log p_{i,y_i} = \frac{1}{N} \sum_{i=1}^{N} \left[\log \sum_{k=0}^{K-1} \exp(z_{ik}) - z_{i,y_i} \right]. \tag{3}$$

The implementation evaluates this loss directly from logits, without a preceding softmax operation. Each optimization step uses the mean loss of its minibatch. Epoch losses weight each minibatch by its sample count, so a smaller final batch does not receive disproportionate weight. The checkpoint with the lowest validation cross-entropy supplies the final test predictions. Test labels do not enter this selection.

Classification performance. Define the confusion matrix C by

$$C_{ab} = \sum_{i=1}^{N} \mathbf{1}\{y_i = a,\ \hat{y}_i = b\}, \tag{4}$$

where rows correspond to true classes and columns to predicted classes. Accuracy is the proportion of correct predictions,

$$\text{Accuracy} = \frac{1}{N} \sum_{k=0}^{K-1} C_{kk}. \tag{5}$$

Accuracy summarizes overall classification performance but does not identify which classes account for the errors.

For class k, let $\mathrm{TP}_k = C_{kk}$, $\mathrm{FP}_k = \sum_{a \neq k} C_{ak}$, and $\mathrm{FN}_k = \sum_{b \neq k} C_{kb}$. We report the unweighted mean of the class F1 scores,

$$F_{1,k} = \frac{2\mathrm{TP}_k}{2\mathrm{TP}_k + \mathrm{FP}_k + \mathrm{FN}_k}, \qquad \text{Macro-F1} = \frac{1}{K} \sum_{k=0}^{K-1} F_{1,k}. \tag{6}$$

A class with a zero denominator receives an F1 score of zero and remains in the average. Macro-F1 gives every class equal weight, irrespective of its frequency. We compute accuracy and macro-F1 from predictions over the complete evaluation split, rather than averaging scores computed separately for each minibatch.

Class-level analysis. Per-class recall measures the fraction of examples of a true class that the model identifies correctly,

$$R_k = \frac{C_{kk}}{\sum_b C_{kb}}. \tag{7}$$

For a class with nonzero support, the row-normalized confusion matrix is $\widetilde{C}_{ab} = C_{ab} / \sum_j C_{aj}$. Its diagonal entries equal class recall; its off-diagonal entries give the fractions assigned to particular incorrect classes. Rows with no true examples are displayed as zero and provide no estimate of recall. Counts and normalized rates are distinguished throughout the error analysis.

Model size and computation. Model size is the number of trainable scalar parameters in the encoder and classification head. This count excludes buffers and optimizer state. It measures model size, not the number of operations required for a prediction. The evaluation code also records the cumulative duration of model forward passes, excluding data loading, input transfer, and subsequent probability and metric computation. These single-pass measurements have no separate warm-up or repeated-trial protocol. They are retained as execution records, not as evidence for a reliable speed ranking across architectures.

2.5 Training configuration

Table 3: Training settings for the reported runs.

Setting	Value
Optimizer	Adam, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$
Learning rate	10^{-3}, constant
Batch size	128
Epochs / seed	50 / 42
Validation fraction	0.1 of the official training split
Weight decay	0
Arithmetic	Float32, no mixed precision
Checkpoint rule	Minimum validation cross-entropy
MLP dropout	0.2 after each hidden activation
Transformer dropout	0.1 in embeddings and encoder layers
CNN normalization	Batch normalization after each convolution

Training uses the dataset-specific transforms shared by all architectures. For MNIST and Fashion-MNIST, pixel values are scaled to $[0, 1]$, randomly rotated within $[-10°, 10°]$, translated by up to 10% of each image dimension, and normalized with mean and standard deviation 0.5. CIFAR-10 uses a random 32×32 crop after four-pixel padding and a random horizontal flip. Its channel means are $(0.4914, 0.4822, 0.4465)$ and standard deviations $(0.2470, 0.2435, 0.2616)$. Validation and test transforms retain scaling and normalization and omit random augmentation.

Saved evaluations record different environments. CNN evaluation uses an NVIDIA GeForce RTX 4060 Ti with Python 3.11.11 and PyTorch 2.14.0+cu126. Other runs include macOS ARM and CUDA environments. These records do not establish comparable conditions for a speed ranking. Augmentation, dropout, and batch normalization also make training and validation losses measurements under different conditions.

2.6 Comparison protocol

The main comparison includes Linear, MLP, CNN, GRU, LSTM, and Transformer on Fashion-MNIST. MNIST is development data and CIFAR-10 is an extension. Only completed evaluations enter the comparisons. Each reported run uses 50 epochs and seed 42. No systematic hyperparameter search or component ablation is documented.

Within each dataset, models share the split, preprocessing, augmentation, optimizer settings, epoch budget, and checkpoint rule. Model-specific regularization and parameter counts remain unequal. Results describe the observed configurations under one seed; repeated runs are required to assess variation across initializations and training randomness.

For detailed Fashion-MNIST error analysis, the flattened-pixel baseline is selected by the lower minimum validation loss among Linear and MLP, with a tie resolved in favour of Linear. The selected baseline is compared with CNN on the same official test images. Confusion matrices and class recall describe all test predictions. Qualitative examples use the first two test indices in each of three categories: CNN correct and baseline wrong, both wrong, and baseline correct and CNN wrong. This rule fixes the examples independently of visual appearance. Category frequencies are computed over the full test set and reported separately from the selected images.

3 Implementation results

3.1 Test performance and model size

The main comparison uses the official Fashion-MNIST test set of 10,000 images. All six models train for 50 epochs with seed 42. Each checkpoint minimizes validation loss; test performance does not enter checkpoint selection.

Table 4: Fashion-MNIST test results. Each model trains for 50 epochs with seed 42; checkpoints minimize validation loss.

Model	Accuracy (%)	Macro-F1	Parameters
Linear	77.05	0.7694	7,850
MLP	87.18	0.8711	235,146
CNN	92.59	0.9261	421,738
GRU	89.88	0.8990	61,962
LSTM	89.52	0.8952	82,186
Transformer	90.43	0.9040	1,191,050

Table 5: Fashion-MNIST checkpoint selection from saved training summaries.

Model	Training epochs	Best epoch	Validation loss
Linear	50	14	0.6540
MLP	50	49	0.3413
CNN	50	42	0.1948
GRU	50	49	0.2676
LSTM	50	46	0.2754
Transformer	50	44	0.2630

CNN reaches 92.59% accuracy and 0.9261 macro-F1. The remaining accuracies are Transformer 90.43%, GRU 89.88%, LSTM 89.52%, MLP 87.18%, Linear 77.05%. CNN exceeds Transformer accuracy by 2.16 percentage points, with 421,738 versus 1,191,050 parameters. These values describe the tested configurations and one seed.

3.2 Learning behavior

The CNN reaches its minimum validation loss at epoch 42, 0.1948. The learning curves compare all six models under the same epoch budget. Training applies augmentation and model-specific regularization; validation does not. A lower validation loss than training loss alone does not diagnose underfitting.

3.3 Per-class performance and error analysis

Class recall is the fraction of correctly classified images within a class. Each Fashion-MNIST test class contains 1,000 images.

CNN Shirt recall is 79.5%, compared with 62.1% for MLP. CNN Coat and Pullover recalls are 86.3% and 90.1%. It assigns 81 Shirt images to T-shirt/top and 61 Coat images to Pullover.

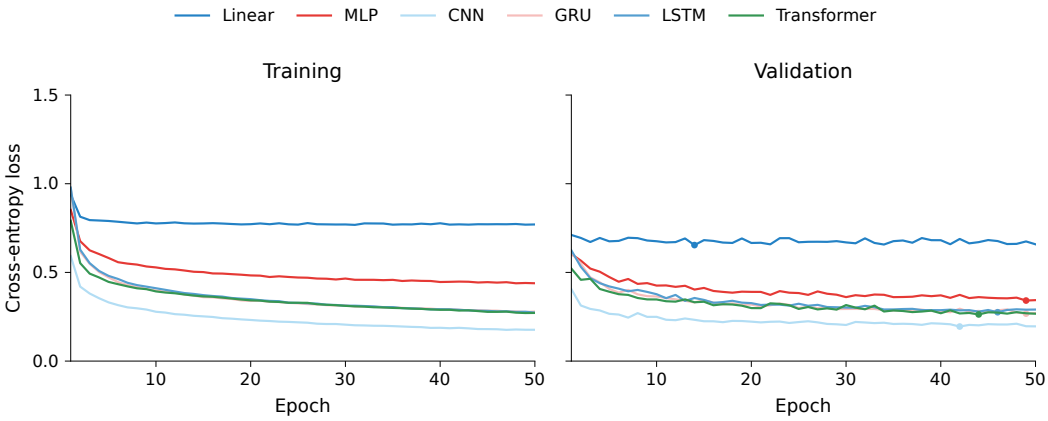
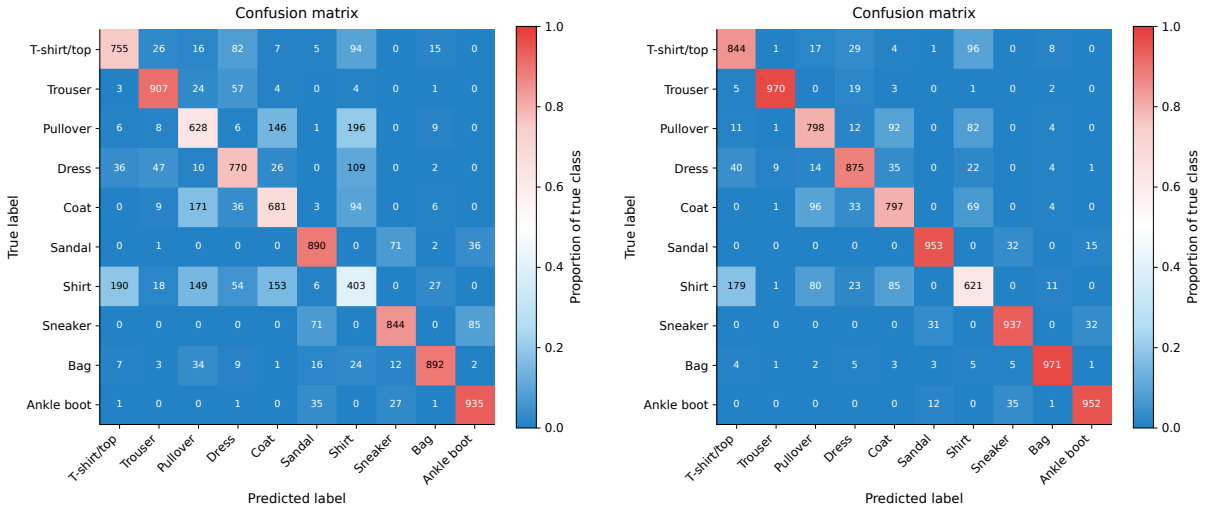


Figure 9: Fashion-MNIST training and validation cross-entropy for six models. Dots mark the selected checkpoints.

Table 6: Fashion-MNIST test recall by class, in percent. Each class contains 1,000 images. All six models use seed 42 and validation-selected checkpoints from 50-epoch runs.

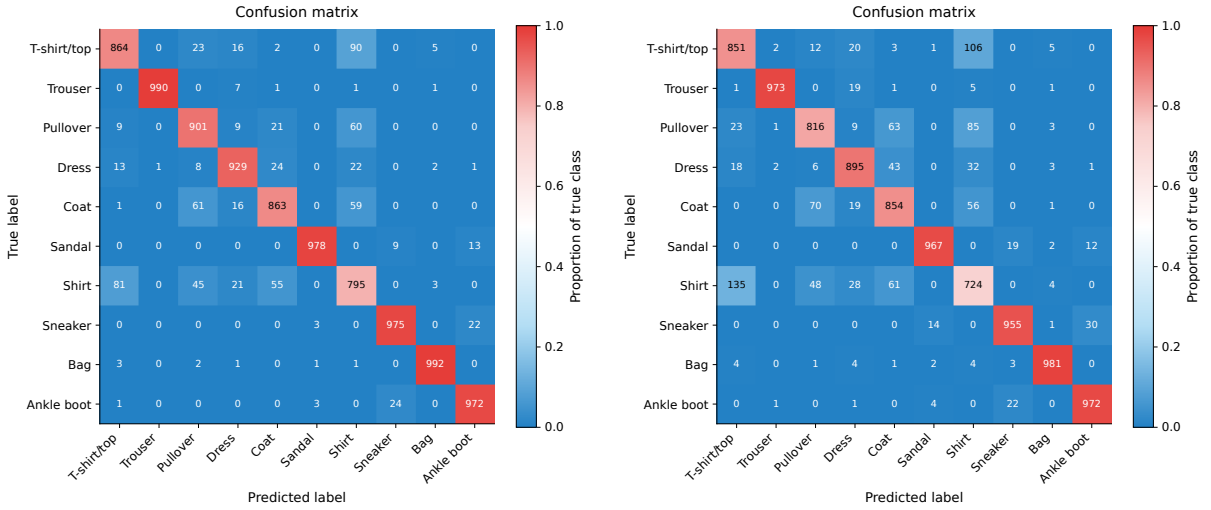
Class	Linear	MLP	CNN	GRU	LSTM	Transformer
T-shirt/top	75.5	84.4	86.4	85.1	83.0	81.8
Trouser	90.7	97.0	99.0	97.3	98.3	98.0
Pullover	62.8	79.8	90.1	81.6	81.6	84.0
Dress	77.0	87.5	92.9	89.5	88.7	93.2
Coat	68.1	79.7	86.3	85.4	84.6	85.6
Sandal	89.0	95.3	97.8	96.7	95.8	96.6
Shirt	40.3	62.1	79.5	72.4	70.9	72.3
Sneaker	84.4	93.7	97.5	95.5	98.7	97.2
Bag	89.2	97.1	99.2	98.1	98.2	98.7
Ankle boot	93.5	95.2	97.2	97.2	95.4	96.9

Errors between T-shirt/top, Pullover, Coat, and Shirt account for 507 of its 741 errors, 68.4%. Both the true and predicted labels must belong to that group for an error to count. Similar outlines are a possible explanation, but these observations do not establish the cause.



Linear MLP

Figure 10: Linear and MLP confusion matrices on the Fashion-MNIST test set. Rows are true classes and columns are predictions. Color shows row proportions; annotations show counts.

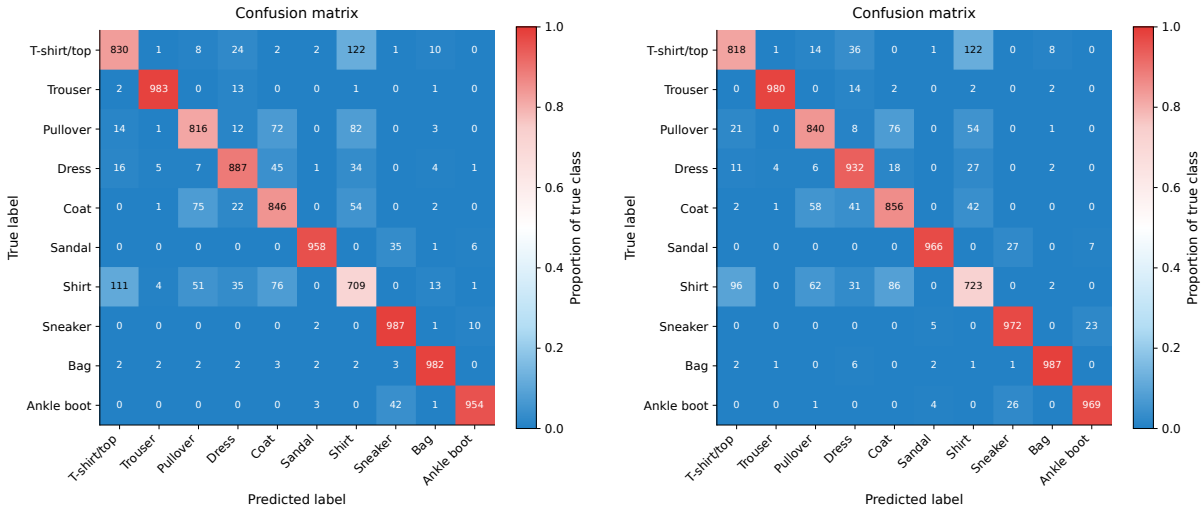


CNN GRU

Figure 11: CNN and GRU confusion matrices on the Fashion-MNIST test set. Rows are true classes and columns are predictions. Color shows row proportions; annotations show counts.

3.4 Predictions on shared images

CNN correctly classifies 775 images that MLP misclassifies. MLP correctly classifies 234 images that CNN misclassifies. Both err on 507 images and both classify 8484 correctly. The disagreement difference is 541 images, consistent with the 5.41 percentage-point accuracy difference. Neither model contains all the other model's correct predictions. The shared examples take the first two official test indices in each of three categories: CNN correct and MLP wrong, both wrong, and MLP correct and CNN wrong. Indices are zero-based. Equal numbers of displayed examples do not represent category frequencies. The figure shows Linear, MLP, CNN, GRU, and Transformer



LSTM Transformer

Figure 12: LSTM and Transformer confusion matrices on the Fashion-MNIST test set. Rows are true classes and columns are predictions. Color shows row proportions; annotations show counts.

predictions. LSTM is included in the aggregate and per-class comparisons; its per-image prediction tensor is not available for this figure.

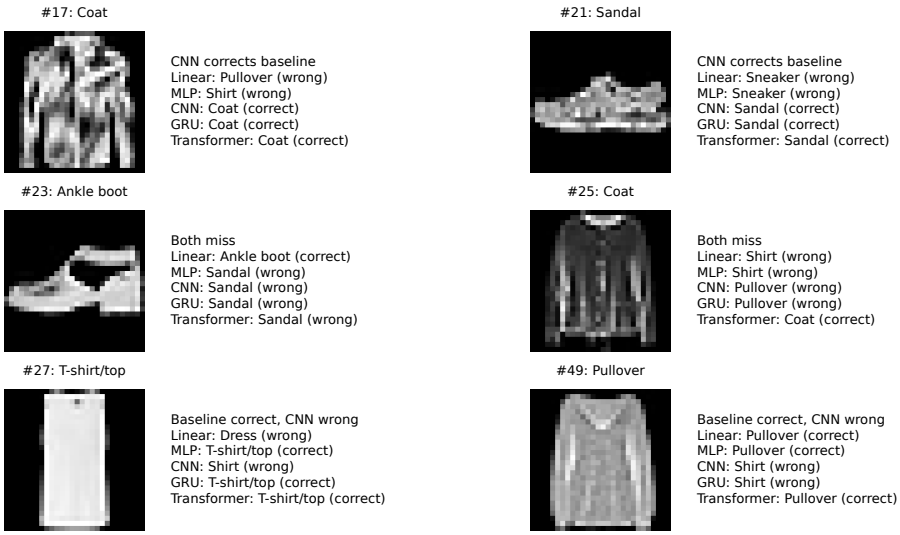


Figure 13: Shared Fashion-MNIST test images selected by CNN/MLP disagreement. Each image has the same five model predictions. Baseline denotes MLP.

3.5 Architecture trade-offs

CNN has the highest Fashion-MNIST test accuracy among the six measured models. The comparison changes architecture, parameter count, and regularization together, so it does not isolate which component explains the result. Recorded training and inference durations use different hardware or timing conditions. They do not support a speed ranking.

3.6 Limitations

Each configuration has one seed, and no systematic tuning study is reported. Accuracy differences therefore do not estimate variability across training runs or establish a general ranking of architecture families. Selected images illustrate errors; they do not measure calibration.

3.7 Conclusion

Preserving spatial features with convolution and a hidden classification layer works well in this Fashion-MNIST comparison. A controlled follow-up should vary the spatial readout while holding training settings fixed and repeat the runs across seeds.

AI usage disclosure

The group used AI tools for project scaffolding, model checks, implementation support, and report editing. The repository's `AI_USAGE.md` records the tools, members, prompts, affected files, and verification for each logged use. On 28 October 2026, Le Nguyen Khang used Codex to audit the classifiers, update the CNN and report comparisons, and edit the report text. A representative request was to replace the CNN description and comparisons with the implemented configuration. Checks used the model code, saved result records, prediction tensors, tests, and a compiled PDF. Final student verification of these edits remains to be completed.

References

[1] Le Thanh Sach, *Course Project Handbook: Assignments 1–3, Deep Learning and Its Applications, CO3133, Semester 261*, HCMUT, revision 14 September 2026. Local source: `docs/handbook-ene-v2.pdf`. The group reports an updated A1 final deadline of 28 October 2026; retain the corresponding LMS announcement as submission evidence.

[2] Group Placeholder, *CO3133 Deep Learning course project repository*. https://github.com/vietanh1802/CO3133-Deep-Learning-261-Placeholder. Source revisions for measured runs are recorded in their test result JSON files.

[3] Han Xiao, Kashif Rasul, and Roland Vollgraf, *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms*, arXiv:1708.07747, 2017. https://arxiv.org/abs/1708.07747.

[4] Zalando Research, *Fashion-MNIST dataset and distribution files*. https://github.com/zalandoresearch/fashion-mnist. Published checksums, label mapping, and repository MIT license; accessed 27 October 2026.

[5] PyTorch contributors, *torchvision.datasets.MNIST documentation*. Online documentation. Accessed 27 October 2026.

[6] Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton, *CIFAR-10 and CIFAR-100 datasets*. https://www.cs.toronto.edu/~kriz/cifar.html. Accessed 27 October 2026.

To complete: Add verified references for the model concepts and library components used in the other sections. Cite them where they support claims. Do not add unread or unverifiable references.